importing the dependemcies

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn.datasets
from sklearn.model_selection import train_test_split
from xgboost import XGBRegressor
from sklearn import metrics
```

importing the Boston House Price Dataset

In [2]:
```
house_price_dataset = sklearn.datasets.load_boston()
```

```
C:\ProgramData\Anaconda3\lib\site-packages\sklearn\utils\deprecation.py:87: FutureWarning: F
unction load_boston is deprecated; `load_boston` is deprecated in 1.0 and will be removed in
1.2.

    The Boston housing prices dataset has an ethical problem. You can refer to
    the documentation of this function for further details.

    The scikit-learn maintainers therefore strongly discourage the use of this
    dataset unless the purpose of the code is to study and educate about
    ethical issues in data science and machine learning.

    In this special case, you can fetch the dataset from the original
    source::

        import pandas as pd
        import numpy as np


        data_url = "http://lib.stat.cmu.edu/datasets/boston"
        raw_df = pd.read_csv(data_url, sep="\s+", skiprows=22, header=None)
        data = np.hstack([raw_df.values[::2, :], raw_df.values[1::2, :2]])
        target = raw_df.values[1::2, 2]

    Alternative datasets include the California housing dataset (i.e.
    :func:`~sklearn.datasets.fetch_california_housing`) and the Ames housing
    dataset. You can load the datasets as follows::

        from sklearn.datasets import fetch_california_housing
        housing = fetch_california_housing()

    for the California housing dataset and::

        from sklearn.datasets import fetch_openml
        housing = fetch_openml(name="house_prices", as_frame=True)

    for the Ames housing dataset.

  warnings.warn(msg, category=FutureWarning)
```

In [3]:

```
1 print(house_price_dataset)
```

{'data': array([[6.3200e-03, 1.8000e+01, 2.3100e+00, ..., 1.5300e+01, 3.9690e+02,
        4.9800e+00],
       [2.7310e-02, 0.0000e+00, 7.0700e+00, ..., 1.7800e+01, 3.9690e+02,
        9.1400e+00],
       [2.7290e-02, 0.0000e+00, 7.0700e+00, ..., 1.7800e+01, 3.9283e+02,
        4.0300e+00],
       ...,
       [6.0760e-02, 0.0000e+00, 1.1930e+01, ..., 2.1000e+01, 3.9690e+02,
        5.6400e+00],
       [1.0959e-01, 0.0000e+00, 1.1930e+01, ..., 2.1000e+01, 3.9345e+02,
        6.4800e+00],
       [4.7410e-02, 0.0000e+00, 1.1930e+01, ..., 2.1000e+01, 3.9690e+02,
        7.8800e+00]]), 'target': array([24. , 21.6, 34.7, 33.4, 36.2, 28.7, 22.9, 27.1, 16.
5, 18.9, 15. ,
       18.9, 21.7, 20.4, 18.2, 19.9, 23.1, 17.5, 20.2, 18.2, 13.6, 19.6,
       15.2, 14.5, 15.6, 13.9, 16.6, 14.8, 18.4, 21. , 12.7, 14.5, 13.2,
       13.1, 13.5, 18.9, 20. , 21. , 24.7, 30.8, 34.9, 26.6, 25.3, 24.7,
       21.2, 19.3, 20. , 16.6, 14.4, 19.4, 19.7, 20.5, 25. , 23.4, 18.9,
       35.4, 24.7, 31.6, 23.3, 19.6, 18.7, 16. , 22.2, 25. , 33. , 23.5,
       19.4, 22. , 17.4, 20.9, 24.2, 21.7, 22.8, 23.4, 24.1, 21.4, 20. ,
       20.8, 21.2, 20.3, 28. , 23.9, 24.8, 22.9, 23.9, 26.6, 22.5, 22.2,
       23.6, 28.7, 22.6, 22. , 22.9, 25. , 20.6, 28.4, 21.4, 38.7, 43.8,
       33.2, 27.5, 26.5, 18.6, 19.3, 20.1, 19.5, 19.5, 20.4, 19.8, 19.4,
       21.7, 22.8, 18.8, 18.7, 18.5, 18.3, 21.2, 19.2, 20.4, 19.3, 22. ,
       20.3, 20.5, 17.3, 18.8, 21.4, 15.7, 16.2, 18. , 14.3, 19.2, 19.6,
       23. , 18.4, 15.6, 18.1, 17.4, 17.1, 13.3, 17.8, 14. , 14.4, 13.4,
       15.6, 11.8, 13.8, 15.6, 14.6, 17.8, 15.4, 21.5, 19.6, 15.3, 19.4,
       17. , 15.6, 13.1, 41.3, 24.3, 23.3, 27. , 50. , 50. , 50. , 22.7,
       25. , 50. , 23.8, 23.8, 22.3, 17.4, 19.1, 23.1, 23.6, 22.6, 29.4,
       23.2, 24.6, 29.9, 37.2, 39.8, 36.2, 37.9, 32.5, 26.4, 29.6, 50. ,
       32. , 29.8, 34.9, 37. , 30.5, 36.4, 31.1, 29.1, 50. , 33.3, 30.3,
       34.6, 34.9, 32.9, 24.1, 42.3, 48.5, 50. , 22.6, 24.4, 22.5, 24.4,
       20. , 21.7, 19.3, 22.4, 28.1, 23.7, 25. , 23.3, 28.7, 21.5, 23. ,
       26.7, 21.7, 27.5, 30.1, 44.8, 50. , 37.6, 31.6, 46.7, 31.5, 24.3,
       31.7, 41.7, 48.3, 29. , 24. , 25.1, 31.5, 23.7, 23.3, 22. , 20.1,
       22.2, 23.7, 17.6, 18.5, 24.3, 20.5, 24.5, 26.2, 24.4, 24.8, 29.6,
       42.8, 21.9, 20.9, 44. , 50. , 36. , 30.1, 33.8, 43.1, 48.8, 31. ,
       36.5, 22.8, 30.7, 50. , 43.5, 20.7, 21.1, 25.2, 24.4, 35.2, 32.4,
       32. , 33.2, 33.1, 29.1, 35.1, 45.4, 35.4, 46. , 50. , 32.2, 22. ,
       20.1, 23.2, 22.3, 24.8, 28.5, 37.3, 27.9, 23.9, 21.7, 28.6, 27.1,
       20.3, 22.5, 29. , 24.8, 22. , 26.4, 33.1, 36.1, 28.4, 33.4, 28.2,
       22.8, 20.3, 16.1, 22.1, 19.4, 21.6, 23.8, 16.2, 17.8, 19.8, 23.1,
       21. , 23.8, 23.1, 20.4, 18.5, 25. , 24.6, 23. , 22.2, 19.3, 22.6,
       19.8, 17.1, 19.4, 22.2, 20.7, 21.1, 19.5, 18.5, 20.6, 19. , 18.7,
       32.7, 16.5, 23.9, 31.2, 17.5, 17.2, 23.1, 24.5, 26.6, 22.9, 24.1,
       18.6, 30.1, 18.2, 20.6, 17.8, 21.7, 22.7, 22.6, 25. , 19.9, 20.8,
       16.8, 21.9, 27.5, 21.9, 23.1, 50. , 50. , 50. , 50. , 50. , 13.8,
       13.8, 15. , 13.9, 13.3, 13.1, 10.2, 10.4, 10.9, 11.3, 12.3, 8.8,
        7.2, 10.5, 7.4, 10.2, 11.5, 15.1, 23.2, 9.7, 13.8, 12.7, 13.1,
       12.5, 8.5, 5. , 6.3, 5.6, 7.2, 12.1, 8.3, 8.5, 5. , 11.9,
       27.9, 17.2, 27.5, 15. , 17.2, 17.9, 16.3, 7. , 7.2, 7.5, 10.4,
        8.8, 8.4, 16.7, 14.2, 20.8, 13.4, 11.7, 8.3, 10.2, 10.9, 11. ,
        9.5, 14.5, 14.1, 16.1, 14.3, 11.7, 13.4, 9.6, 8.7, 8.4, 12.8,
       10.5, 17.1, 18.4, 15.4, 10.8, 11.8, 14.9, 12.6, 14.1, 13. , 13.4,
       15.2, 16.1, 17.8, 14.9, 14.1, 12.7, 13.5, 14.9, 20. , 16.4, 17.7,
       19.5, 20.2, 21.4, 19.9, 19. , 19.1, 19.1, 20.1, 19.9, 19.6, 23.2,
       29.8, 13.8, 13.3, 16.7, 12. , 14.6, 21.4, 23. , 23.7, 25. , 21.8,
       20.6, 21.2, 19.1, 20.6, 15.2, 7. , 8.1, 13.6, 20.1, 21.8, 24.5,
       23.1, 19.7, 18.3, 21.2, 17.5, 16.8, 22.4, 20.6, 23.9, 22. , 11.9]), 'feature_names':
array(['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD',
       'TAX', 'PTRATIO', 'B', 'LSTAT'], dtype='<U7'), 'DESCR': ".. _boston_dataset:\n\nBosto
n house prices dataset\n---------------------------\n\n**Data Set Characteristics:** \n\n
:Number of Instances: 506 \n\n   :Number of Attributes: 13 numeric/categorical predictive.
Median Value (attribute 14) is usually the target.\n\n   :Attribute Information (in orde
r):\n   - CRIM   per capita crime rate by town\n   - ZN   proportion of resi
dential land zoned for lots over 25,000 sq.ft.\n   - INDUS   proportion of non-retail
business acres per town\n   - CHAS   Charles River dummy variable (= 1 if tract bound
s river; 0 otherwise)\n   - NOX   nitric oxides concentration (parts per 10 million)
\n   - RM   average number of rooms per dwelling\n   - AGE   proportion of
owner-occupied units built prior to 1940\n   - DIS   weighted distances to five Bost
on employment centres\n   - RAD   index of accessibility to radial highways\n
- TAX   full-value property-tax rate per $10,000\n   - PTRATIO pupil-teacher ratio
by town\n   - B   1000(Bk - 0.63)^2 where Bk is the proportion of black people by
town\n   - LSTAT   % lower status of the population\n   - MEDV   Median value o

```
f owner-occupied homes in $1000's\n\n    :Missing Attribute Values: None\n\n    :Creator: Ha
rrison, D. and Rubinfeld, D.L.\n\nThis is a copy of UCI ML housing dataset.\nhttps://archiv
e.ics.uci.edu/ml/machine-learning-databases/housing/\n\n\nThis dataset was taken from the St
atLib library which is maintained at Carnegie Mellon University.\n\nThe Boston house-price d
ata of Harrison, D. and Rubinfeld, D.L. 'Hedonic\nprices and the demand for clean air', J. E
nviron. Economics & Management,\nvol.5, 81-102, 1978.   Used in Belsley, Kuh & Welsch, 'Regr
ession diagnostics\n...', Wiley, 1980.   N.B. Various transformations are used in the table
on\npages 244-261 of the latter.\n\nThe Boston house-price data has been used in many machin
e learning papers that address regression\nproblems.   \n     \n.. topic:: References\n\n
- Belsley, Kuh & Welsch, 'Regression diagnostics: Identifying Influential Data and Sources o
f Collinearity', Wiley, 1980. 244-261.\n   - Quinlan,R. (1993). Combining Instance-Based and
Model-Based Learning. In Proceedings on the Tenth International Conference of Machine Learni
ng, 236-243, University of Massachusetts, Amherst. Morgan Kaufmann.\n", 'filename': 'boston_
house_prices.csv', 'data_module': 'sklearn.datasets.data'}
```

In [4]:
```
# loading the dataset to a pandas DataFrame
house_price_dataframe = pd.DataFrame(house_price_dataset.data, columns = house_price_data
```

In [5]:
```
# print first 5 rows of our dtaframe
house_price_dataframe.head()
```

Out[5]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT
0	0.00632	18.0	2.31	0.0	0.538	6.575	65.2	4.0900	1.0	296.0	15.3	396.90	4.98
1	0.02731	0.0	7.07	0.0	0.469	6.421	78.9	4.9671	2.0	242.0	17.8	396.90	9.14
2	0.02729	0.0	7.07	0.0	0.469	7.185	61.1	4.9671	2.0	242.0	17.8	392.83	4.03
3	0.03237	0.0	2.18	0.0	0.458	6.998	45.8	6.0622	3.0	222.0	18.7	394.63	2.94
4	0.06905	0.0	2.18	0.0	0.458	7.147	54.2	6.0622	3.0	222.0	18.7	396.90	5.33

In [6]:
```
# add the target (price) column to the DataFrame
house_price_dataframe['price'] = house_price_dataset.target
```

In [7]:
```
house_price_dataframe.head()
```

Out[7]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	price
0	0.00632	18.0	2.31	0.0	0.538	6.575	65.2	4.0900	1.0	296.0	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0.0	0.469	6.421	78.9	4.9671	2.0	242.0	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0.0	0.469	7.185	61.1	4.9671	2.0	242.0	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0.0	0.458	6.998	45.8	6.0622	3.0	222.0	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0.0	0.458	7.147	54.2	6.0622	3.0	222.0	18.7	396.90	5.33	36.2

In [8]:
```
#numbers of rows and colums
house_price_dataframe.shape
```

Out[8]: (506, 14)

In [9]:
```
# number of missing values
house_price_dataframe.isnull().sum()

```

Out[9]:
```
CRIM       0
ZN         0
INDUS      0
CHAS       0
NOX        0
RM         0
AGE        0
DIS        0
RAD        0
TAX        0
PTRATIO    0
B          0
LSTAT      0
price      0
dtype: int64
```

In [10]:
```
# statistical measures of the dataset
house_price_dataframe.describe()
```

Out[10]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD
count	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000
mean	3.613524	11.363636	11.136779	0.069170	0.554695	6.284634	68.574901	3.795043	9.549407
std	8.601545	23.322453	6.860353	0.253994	0.115878	0.702617	28.148861	2.105710	8.707259
min	0.006320	0.000000	0.460000	0.000000	0.385000	3.561000	2.900000	1.129600	1.000000
25%	0.082045	0.000000	5.190000	0.000000	0.449000	5.885500	45.025000	2.100175	4.000000
50%	0.256510	0.000000	9.690000	0.000000	0.538000	6.208500	77.500000	3.207450	5.000000
75%	3.677083	12.500000	18.100000	0.000000	0.624000	6.623500	94.075000	5.188425	24.000000
max	88.976200	100.000000	27.740000	1.000000	0.871000	8.780000	100.000000	12.126500	24.000000

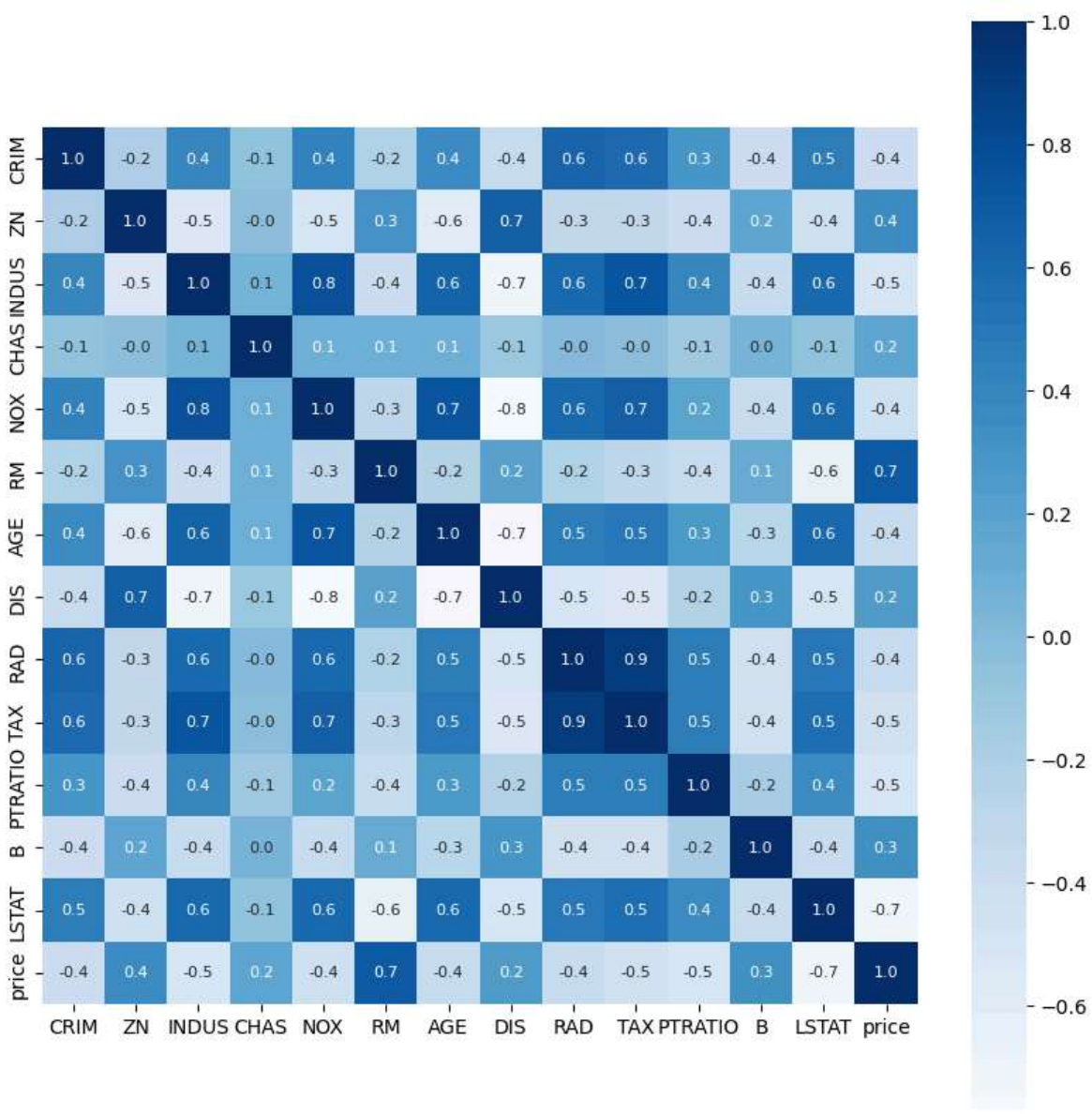
understanding the correlation between various features and in the dataset

In [11]:
```
correlation = house_price_dataframe.corr()
```

In [12]:

```
# constructing a heatmap to understand the correlation
plt.figure(figsize=(10,10))
sns.heatmap(correlation, cbar=True, square=True, fmt='.1f', annot=True, annot_kws={'size'
```

Out[12]: <AxesSubplot:>



splitting the data and Target

In [13]:

```
x = house_price_dataframe.drop(['price'], axis=1)
y = house_price_dataframe['price']
```

In [14]:
```
print(x)
print(y)
```

```
        CRIM    ZN  INDUS  CHAS    NOX     RM   AGE     DIS  RAD    TAX  \
0    0.00632  18.0   2.31   0.0  0.538  6.575  65.2  4.0900  1.0  296.0
1    0.02731   0.0   7.07   0.0  0.469  6.421  78.9  4.9671  2.0  242.0
2    0.02729   0.0   7.07   0.0  0.469  7.185  61.1  4.9671  2.0  242.0
3    0.03237   0.0   2.18   0.0  0.458  6.998  45.8  6.0622  3.0  222.0
4    0.06905   0.0   2.18   0.0  0.458  7.147  54.2  6.0622  3.0  222.0
..       ...   ...    ...   ...    ...    ...   ...     ...  ...    ...
501  0.06263   0.0  11.93   0.0  0.573  6.593  69.1  2.4786  1.0  273.0
502  0.04527   0.0  11.93   0.0  0.573  6.120  76.7  2.2875  1.0  273.0
503  0.06076   0.0  11.93   0.0  0.573  6.976  91.0  2.1675  1.0  273.0
504  0.10959   0.0  11.93   0.0  0.573  6.794  89.3  2.3889  1.0  273.0
505  0.04741   0.0  11.93   0.0  0.573  6.030  80.8  2.5050  1.0  273.0

     PTRATIO       B  LSTAT
0       15.3  396.90   4.98
1       17.8  396.90   9.14
2       17.8  392.83   4.03
3       18.7  394.63   2.94
4       18.7  396.90   5.33
..       ...     ...    ...
501     21.0  391.99   9.67
502     21.0  396.90   9.08
503     21.0  396.90   5.64
504     21.0  393.45   6.48
505     21.0  396.90   7.88

[506 rows x 13 columns]
0      24.0
1      21.6
2      34.7
3      33.4
4      36.2
       ...
501    22.4
502    20.6
503    23.9
504    22.0
505    11.9
Name: price, Length: 506, dtype: float64
```

splitting the data into Training data and Test data

In [15]:
```
x_train, x_test, y_train, y_test = train_test_split(x,y, test_size = 0.2, random_state =
```

In [16]:
```
print(x.shape, x_train.shape, x_test.shape)
```

```
(506, 13) (404, 13) (102, 13)
```

Model Training

XGBoost Regressor

In [17]:
```
# loading the model
model = XGBRegressor()
```

In [18]:
```
# training the model with x_train
model.fit(x_train, y_train)
```

Out[18]:
```
XGBRegressor(base_score=None, booster=None, callbacks=None,
             colsample_bylevel=None, colsample_bynode=None,
             colsample_bytree=None, device=None, early_stopping_rounds=None,
             enable_categorical=False, eval_metric=None, feature_types=None,
             gamma=None, grow_policy=None, importance_type=None,
             interaction_constraints=None, learning_rate=None, max_bin=None,
             max_cat_threshold=None, max_cat_to_onehot=None,
             max_delta_step=None, max_depth=None, max_leaves=None,
             min_child_weight=None, missing=nan, monotone_constraints=None,
             multi_strategy=None, n_estimators=None, n_jobs=None,
             num_parallel_tree=None, random_state=None, ...)
```

Evaluation

Prediction on training data

In [19]:
```
# accuracy for prediction on training data
training_data_prediction = model.predict(x_train)
```

In [20]:

```
1 print(training_data_prediction)
```

```
[23.112196  20.992601  20.10438   34.67932   13.920501  13.499354
 21.998383  15.206723  10.89543   22.67402   13.795236   5.602332
 29.808502  49.98666   34.89634   20.594336  23.388903  19.2118
 32.69294   19.604128  26.978151   8.405952  46.00062   21.70406
 27.084402  19.372278  19.297894  24.79984   22.608278  31.707775
 18.53683    8.703393  17.40025   23.698814  13.29729   10.504759
 12.693588  24.994888  19.694864  14.911037  24.20254   24.991112
 14.901547  16.987965  15.592753  12.704759  24.505623  15.007718
 49.999355  17.509344  21.18844   31.999287  15.606071  22.902134
 19.309835  18.697083  23.302961  37.19767   30.102247  33.117855
 20.993683  50.00471   13.40048    5.002565  16.50862    8.4016905
 28.651423  19.49218   20.595366  45.404697  39.808857  33.4055
 19.81498   33.406376  25.30206   49.998615  12.544487  17.433802
 18.602612  22.601418  50.004013  23.814182  23.313164  23.097467
 41.71243   16.112017  31.604454  36.09397    7.0009975 20.406271
 19.992195  12.003392  25.027754  49.98552   37.890903  23.091173
 41.289513  17.604618  16.30125   30.05175   22.884857  19.802671
 17.106977  18.903633  18.897047  22.598665  23.170893  33.19197
 15.00434   11.704804  18.795511  20.817484  17.998543  19.633396
 49.998672  17.208574  16.410513  17.506626  14.6008    33.09849
 14.504811  43.813366  34.900055  20.388191  14.605566   8.091776
 11.777508  11.811628  18.691      6.322443  23.97163   13.073076
 19.595     49.99033   22.319597  18.91175   31.203646  20.712711
 32.200443  36.188755  14.222898  15.705663  50.000664  20.408077
 16.185907  13.410434  50.012474  31.60327   12.288182  19.18906
 29.809902  31.49241   22.804003  10.194443  24.09609   23.705154
 22.008154  13.790835  28.399841  33.199585  13.102867  19.017357
 26.61559   36.963135  30.7939    22.80785   10.206419  22.19713
 24.482466  36.19345   23.092129  20.12124   19.498154  10.796299
 22.701403  19.49908   20.107922   9.625605  42.797676  48.79655
 13.099009  20.29537   24.794712  14.106459  21.698246  22.188694
 32.99889   21.09952   24.998121  19.110165  32.401157  13.601795
 15.072056  23.06062   27.487326  19.401924  26.481848  27.50343
 28.686726  21.19214   18.701029  26.7093    14.01264   21.699009
 18.39739   43.11556   29.09378   20.298742  23.711458  18.30434
 17.193619  18.321108  24.392206  26.391497  19.10248   13.302614
 22.189732  22.199099   8.530714  18.889635  21.800455  19.305798
 18.198288   7.4938145 22.400797  20.028303  14.404203  22.500402
 28.504164  21.608568  13.798578  20.495127  21.902288  23.100073
 50.00128   16.23443   30.298399  49.996014  17.78638   19.060133
 10.39715   20.383387  16.496948  17.195917  16.681927  19.509869
 30.502445  29.01701   19.558786  23.172018  24.397314   9.528121
 23.894762  49.996834  21.196695  22.596247  19.989746  13.393513
 19.995872  17.068512  12.718964  23.01111   15.199219  20.609226
 26.19055   18.109114  24.098877  14.100204  21.695303  20.096022
 25.018776  27.899471  22.918222  18.499252  22.202477  23.99494
 14.8048935 19.896328  24.411158  17.790047  24.596226  32.007046
 17.778685  23.309103  16.120615  13.003008  10.993355  24.306978
 15.597863  35.20248   19.58716   42.29605    8.789314  24.399925
 14.109244  15.4010315 17.299047  22.113592  23.106049  44.805172
 17.795519  31.499706  22.813938  16.836212  23.911596  12.09551
 38.69628   21.387049  16.001123  23.929094  11.897898  24.983562
  7.1969633 24.69086   18.187803  22.471941  23.013317  24.295506
 17.099222  17.796907  13.503164  27.094381  13.296886  21.90404
 19.99361   15.402385  16.588629  22.29326   24.697983  21.428938
 22.882269  29.601665  21.881992  19.908726  29.60596   23.408524
 13.807421  24.499699  11.901903   7.20547   20.484905   9.706262
 48.301437  25.194635  11.691466  17.39672   14.49594   28.584557
 19.395731  22.486904   7.0219784 20.60076   22.998001  19.699215
 23.700571  25.02278   27.992222  13.39496   14.524017  20.30391
 19.304321  24.108646  14.88511   26.387497  33.31608   23.61982
 24.60193   18.494753  20.90211   10.411172  23.305649  13.097067
 24.699335  22.610847  20.50208   16.82098   10.198874  33.805454
 18.60289   50.0009    23.778967  23.91014   21.15922   18.81689
  8.491747  21.506403  23.200815  21.043766  16.604784  28.060492
 21.197857  28.370916  14.2918625 49.997353  30.989647  24.980095
 21.410505  19.000553  29.00484   15.204052  22.791481  21.791014
 19.896528  23.77255  ]
```

In [21]:
```
# R squared error
score_1 = metrics.r2_score(y_train, training_data_prediction)

# mean absolute error
score_2 = metrics.mean_absolute_error(y_train, training_data_prediction)

print("R squared error : ", score_1)
print("Mean Absolute error : ", score_2)
```

```
R squared error :  0.9999980039471451
Mean Absolute error :  0.0091330346494618
```

Visualizing the actual Prices and predicted prices

In [22]:
```
plt.scatter(y_train, training_data_prediction)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual Price vs Predicted Price")
plt.show()
```

Prediction on test data

In [23]:
```
# accuracy for prediction on test data
test_data_prediction = model.predict(x_test)
```

In [24]:
```
# R squared error
score_1 = metrics.r2_score(y_test, test_data_prediction)

# mean absolute error
score_2 = metrics.mean_absolute_error(y_test, test_data_prediction)

print("R squared error : ", score_1)
print("Mean Absolute error : ", score_2)
```

```
R squared error :  0.9051721149855378
Mean Absolute error :  2.0748727686264927
```

In [ ]:
```

```